listing 6.4, an variant, `searchNode`, that returns the node found, rather than just `true` or `false`: we'll see in the next sections already how this can be used to implement `remove`.

Performance-wise, how fast is `search`? The number of recursive calls we make is limited by the smaller of two values: the maximum height of the trie and the length of the search string. The latter is usually shorter than the former, but either way, for a string of length `m` we can be sure that no more than `O(m)` calls are going to be made, independently on the number of keys stored in the trie.

The key is, then, how long each step takes: turns out, there are three factors influencing this cost:

1. The cost of comparing 2 characters: this can be assumed to be `O(1)`.
2. The cost of finding the next node: for an alphabet ∑ of size `k`, this can be, depending on the implementation:

   a) Constant (amortized or worst-case[13]), `O(1)`, using hash tables for edges;

   b) Logarithmic worst-case `O(log(k))`, using a balanced tree.

   c) Linear worst-case `O(k)`, using plain arrays.

   Amortized constant time can be reasonably be assumed in most cases.
3. The cost of following a link and of splitting a string into `head+tail`. This point is the one where we need to be really careful. The naïve approach of extracting a substring for each node would be a performance disaster in most programming languages. Strings are